

BUỔI 6 - Giao diện Danh sách Môn học, Tìm kiếm & Phân trang

Điều kiện tiên quyết: đã hoàn thành Buổi 5 - *crs-frontend* đã có *axiosClient* trở về Gateway, *types/course.ts*, *api/courseApi.ts*, và *App.tsx* tạm thời gọi được *GET /api/courses*.
Buổi này thay thế hoàn toàn component test tạm ở *App.tsx* bằng giao diện thật, tái sử dụng nguyên vẹn *axiosClient* và *types/course.ts* đã viết ở Buổi 5.

0. Mục tiêu buổi học

Sau buổi học này, sinh viên phải:

- Tách được component theo đúng trách nhiệm: **CourseList** (hiển thị), **SearchBox** (nhập từ khoá), **Pagination** (điều hướng trang) - không dồn hết logic vào 1 component khổng lồ.
- Gọi **GET /api/courses** qua Gateway với tham số **keyword**, **page**, **size** - đúng như API đã hoàn thiện ở Buổi 3.
- Xử lý đầy đủ **4 trạng thái bắt buộc** của mọi màn hình gọi API: **Loading**, **Success**, **Empty**, **Error** - đây là yêu cầu xuyên suốt cho mọi màn hình từ buổi này đến hết.

A. Vì sao phải xử lý đủ 4 trạng thái?

Quan sát bảng:

Trạng thái	Khi nào xảy ra	Nếu bỏ qua sẽ gây ra vấn đề gì
Loading	Đang chờ phản hồi từ Gateway	Người dùng thấy màn hình trống, tưởng ứng dụng bị treo, có thể bấm lại nhiều lần
Success	Có dữ liệu, mảng không rỗng	(Trạng thái "bình thường" — dễ nhớ nhất, ít bị quên nhất)
Empty	Gọi thành công nhưng mảng rỗng (ví dụ search không ra kết quả)	Nếu không xử lý riêng, người dùng thấy bảng trống trơn, tưởng lỗi hệ thống thay vì hiểu là "không tìm thấy"
Error	Lỗi mạng, lỗi 500, Gateway/service down	Nếu không xử lý, React có thể crash trắng trang hoặc hiển thị dữ liệu cũ gây hiểu lầm

Đây chính là 4 trạng thái đã ghi trong outline Buổi 6 gốc và sẽ được kiểm tra lại trong checklist tổng kết

ở Buổi sau ("Giao diện xử lý đủ Loading/Success/Empty/Error, kể cả lỗi lan truyền từ service khác").

B. Custom hook useCourses - tách logic gọi API ra khỏi component hiển thị

B.1. Vì sao dùng custom hook thay vì gọi API thẳng trong CourseList?

Nếu viết useEffect gọi API trực tiếp trong CourseList, component vừa phải lo hiển thị vừa phải lo quản lý state gọi API - khó tái sử dụng. Tách ra useCourses giúp CourseList chỉ tập trung vào việc render UI dựa trên state đã có sẵn.

```
// path: crs-frontend/src/api/useCourses.ts
// purpose: custom hook quản lý việc gọi GET /api/courses (search +
// phân trang) và 4 trạng thái
// Loading/Success/Empty/Error, tách riêng khỏi component hiển thị

import { useState, useEffect, useCallback } from 'react';
import { getCourses } from '../courseApi';
import type { Course } from '../types/course';
import type { ApiErrorResponse } from '../types/apiError';
import axios from 'axios';

export type LoadState = 'loading' | 'success' | 'empty' | 'error';

export function useCourses(keyword: string, page: number, size = 10)
{
  const [courses, setCourses] = useState<Course[]>([]);
  const [totalPages, setTotalPages] = useState(0);
  const [state, setState] = useState<LoadState>('loading');
  const [errorMessage, setErrorMessage] = useState<string>('');

  const fetchCourses = useCallback(() => {
    setState('loading');
    getCourses(keyword, page, size)
      .then((res) => {
        const data = res.data;
        setCourses(data.content);
      })
      .catch((err) => {
        setErrorMessage(err.message);
        setState('error');
      });
  }, [keyword, page, size]);

  useEffect(() => {
    fetchCourses();
  }, [keyword, page, size]);
}
```

```

        setTotalPages(data.totalPages);
        setState(data.content.length === 0 ? 'empty' : 'success');
    })
    .catch((err) => {
        let message = 'Da xay ra loi khong xac dinh, vui long thu
lai.';
        if (axios.isAxiosError<ApiErrorResponse>(err)) {
            if (err.response?.data?.message) {
                message = err.response.data.message;
            } else if (!err.response) {
                // Khong nhan duoc response nao ca - Gateway hoac
course-service dang tat
                message = 'Khong ket noi duoc toi he thong. Vui long thu
lai sau.';
            }
        }
        setErrorMessage(message);
        setState('error');
    });
}, [keyword, page, size]);

useEffect(() => {
    fetchCourses();
}, [fetchCourses]);

return { courses, totalPages, state, errorMessage, refetch:
fetchCourses };
}

```

Nhắc lại kiến thức Buổi 5: nhánh !err.response (không nhận được response nào) chính là tình huống sẽ được mở xẻ sâu hơn ở Buổi sau khi thử tắt hẳn một service - Frontend phải phân biệt được "lỗi nghiệp vụ có message rõ ràng từ backend" và "lỗi kết nối hoàn toàn không có phản hồi".

C. Component SearchBox - có debounce

C.1. Vì sao cần debounce?

Nếu gọi API ngay mỗi lần người dùng gõ 1 ký tự, hệ thống sẽ gửi rất nhiều request thừa (gõ "lap trinh" = 9 request). Debounce giúp chỉ gọi API sau khi người dùng ngừng gõ một khoảng thời gian ngắn (thường 400–500ms).

```
// path: crs-frontend/src/components/SearchBox.tsx
// purpose: o nhap tim kiem mon hoc, co debounce de tranh goi API
lien tuc khi go phim

import { useState, useEffect } from 'react';

interface SearchBoxProps {
  onSearch: (keyword: string) => void;
  placeholder?: string;
}

export default function SearchBox({ onSearch, placeholder }:
SearchBoxProps) {
  const [inputValue, setInputValue] = useState('');

  useEffect(() => {
    const timer = setTimeout(() => {
      onSearch(inputValue.trim());
    }, 400);

    return () => clearTimeout(timer); // huy timer cu moi lan
    inputValue thay doi
  }, [inputValue, onSearch]);

  return (
    <input
      type="text"
      value={inputValue}
      onChange={(e) => setInputValue(e.target.value)}
      placeholder={placeholder ?? 'Tim kiem theo ten mon hoc...'}
      style={{
```

```

        width: '100%',
        maxWidth: 400,
        padding: '8px 12px',
        fontSize: 14,
        border: '1px solid #ccc',
        borderRadius: 6,
      }}
    />
  );
}

```

D. Component Pagination

```

// path: crs-frontend/src/components/Pagination.tsx
// purpose: điều hướng trang, dùng chung cho mọi màn hình danh sách
// có phân trang trong hệ thống

interface PaginationProps {
  currentPage: number; // bắt đầu từ 0, dùng định dạng giống Spring
  totalPages: number;
  onChange: (page: number) => void;
}

export default function Pagination({ currentPage, totalPages,
  onChange }: PaginationProps) {
  if (totalPages <= 1) return null;

  const pages = Array.from({ length: totalPages }, (_, i) => i);

  return (
    <div style={{ display: 'flex', gap: 6, marginTop: 16 }}>
      <button
        disabled={currentPage === 0}
        onClick={() => onChange(currentPage - 1)}
      >

```

```

    « Trang truoc
  </button>

  {pages.map((p) => (
    <button
      key={p}
      onClick={() => onPageChange(p)}
      style={{
        fontWeight: p === currentPage ? 'bold' : 'normal',
        textDecoration: p === currentPage ? 'underline' :
'none',
      }}
    >
      {p + 1}
    </button>
  ))}

  <button
    disabled={currentPage >= totalPages - 1}
    onClick={() => onPageChange(currentPage + 1)}
  >
    Trang sau »
  </button>
</div>
);
}

```

Với danh sách môn học thực tế có thể lên tới hàng chục trang, đây là bản đơn giản hiển thị hết số trang; sinh viên có thể tự nâng cấp thành dạng rút gọn (1 2 3 ... 8 9 10) như bài tập mở rộng.

E. Component CourseList — ráp 4 trạng thái

```

// path: crs-frontend/src/components/CourseList.tsx
// purpose: hiển thị danh sách môn học, xử lý 4 trạng thái
Loading/Success/Empty/Error

```

```

import type { Course } from '../types/course';
import type { LoadState } from '../api/useCourses';

interface CourseListProps {
  courses: Course[];
  state: LoadState;
  errorMessage: string;
  onRetry: () => void;
}

export default function CourseList({ courses, state, errorMessage,
onRetry }: CourseListProps) {
  if (state === 'loading') {
    return <p>Dang tai danh sach mon hoc...</p>;
  }

  if (state === 'error') {
    return (
      <div style={{ color: '#b91c1c' }}>
        <p>{errorMessage}</p>
        <button onClick={onRetry}>Thu lai</button>
      </div>
    );
  }

  if (state === 'empty') {
    return <p>Khong tim thay mon hoc nao phu hop.</p>;
  }

  // state === 'success'
  return (
    <table style={{ width: '100%', borderCollapse: 'collapse' }}>
      <thead>
        <tr style={{ textAlign: 'left', borderBottom: '2px solid
#333' }}>
          <th>Ten mon hoc</th>
          <th>So tin chi</th>

```

```

        <th>Số chỗ còn lại</th>
      </tr>
    </thead>
    <tbody>
      {courses.map((course) => (
        <tr key={course.id} style={{ borderBottom: '1px solid
#eee' }}>
          <td>{course.tenMonHoc}</td>
          <td>{course.soTinChi}</td>
          <td style={{ color: course.soChoConLai === 0 ? '#b91c1c'
: 'inherit' }}>
            {course.soChoConLai} / {course.soChoToiDa}
          </td>
        </tr>
      ))}
    </tbody>
  </table>
);
}

```

Cột "Số chỗ còn lại" tô đỏ khi bằng 0 - chi tiết nhỏ này sẽ có ý nghĩa thực tế hơn ở Buổi sau, khi sinh viên thêm nút "Đăng ký" và cần disable nút khi hết chỗ.

F. Ráp lại App.tsx - thay thế component test tạm của Buổi 5

```

// path: crs-frontend/src/App.tsx
// purpose: trang danh sách môn học hoàn chỉnh, thay thế component
test tạm của Buổi 5,
// phối hợp SearchBox + CourseList + Pagination + useCourses

import { useState } from 'react';
import { useCourses } from '../api/useCourses';
import SearchBox from '../components/SearchBox';
import CourseList from '../components/CourseList';

```

```

import Pagination from './components/Pagination';

function App() {
  const [keyword, setKeyword] = useState('');
  const [page, setPage] = useState(0);

  const { courses, totalPages, state, errorMessage, refetch } =
useCourses(keyword, page);

  const handleSearch = (newKeyword: string) => {
    setKeyword(newKeyword);
    setPage(0); // moi lan tim kiem moi, luon quay ve trang dau
  };

  return (
    <div style={{ padding: 24, fontFamily: 'sans-serif', maxWidth:
800, margin: '0 auto' }}>
      <h1>Danh sach mon hoc</h1>
      <SearchBox onSearch={handleSearch} />
      <div style={{ marginTop: 16 }}>
        <CourseList
          courses={courses}
          state={state}
          errorMessage={errorMessage}
          onRetry={refetch}
        />
      </div>
      <Pagination currentPage={page} totalPages={totalPages}
onPageChange={setPage} />
    </div>
  );
}

export default App;

```

G. Test thủ công theo kịch bản đủ 4 trạng thái

#	Thao tác	Trạng thái cần quan sát
1	Mở lại trang (F5) khi mạng bình thường	Thấy dòng "Đang tải..." thoáng qua, sau đó bảng dữ liệu hiện ra
2	Gõ vào ô tìm kiếm một từ khoá chắc chắn không khớp môn nào (ví dụ "zzz123")	Sau ~400ms, hiện đúng thông báo "Không tìm thấy môn học nào phù hợp"
3	Xoá từ khoá tìm kiếm	Danh sách quay lại đầy đủ, quay về trang 1
4	Bấm chuyển trang (nếu có ≥ 2 trang dữ liệu - có thể cần tạo thêm môn học qua Postman/Buổi 2 để đủ dữ liệu test)	Danh sách đổi đúng theo trang, số trang đang chọn được in đậm
5	Tắt hẳn api-gateway trong IntelliJ, sau đó bấm nút "Thử lại" trên giao diện	Hiện đúng thông báo lỗi kết nối, có nút "Thử lại", không crash trắng trang
6	Bật lại api-gateway, bấm "Thử lại"	Danh sách tải lại thành công

H. Git commit

```
cd crs-microservices
git add crs-frontend/
git commit -m "feat: course list with search + pagination via gateway"
git push
```

Sản phẩm đầu ra cuối buổi

- ☑ useCourses tách riêng logic gọi API khỏi component hiển thị.
- ☑ SearchBox có debounce, không gọi API dồn dập khi gõ phím.
- ☑ Pagination hoạt động đúng, đồng bộ với tham số page gửi lên GET /api/courses.
- ☑ CourseList xử lý đủ và đúng cả 4 trạng thái Loading/Success/Empty/Error theo đúng kịch bản test ở mục G.
- ☑ Tìm kiếm mới luôn quay về trang đầu tiên (page = 0), không giữ nguyên trang cũ gây hiểu nhầm "không có kết quả".
- ☑ Git có commit feat: course list with search + pagination via gateway.

Lỗi thường gặp & cách xử lý nhanh

Triệu chứng	Nguyên nhân thường gặp	Cách xử lý
Gõ tìm kiếm bị giật, gọi API liên tục mỗi ký tự	Quên clearTimeout trong useEffect của SearchBox, hoặc đặt debounce logic sai vị trí (trong useCourses thay vì SearchBox)	Xem lại mục C, đảm bảo hàm return của useEffect có clearTimeout
Đổi từ khoá tìm kiếm nhưng danh sách vẫn hiện trang 3/4 trống trơn	Quên setPage(0) trong handleSearch	Xem lại mục F
Trạng thái luôn dừng ở "loading", không bao giờ chuyển	Thiếu .catch() hoặc quên gọi setState trong nhánh lỗi của useCourses	Xem lại mục B.1
Pagination hiện cả khi chỉ có 1 trang dữ liệu	Thiếu điều kiện if (totalPages <= 1) return null;	Xem lại mục D
Trạng thái Error hiển thị message kỹ thuật khó hiểu (ví dụ "Network Error" tiếng Anh thô)	Chưa map đúng nhánh !err.response trong useCourses	Xem lại mục B.1, đối chiếu với kịch bản test #5 ở mục G